

消息中间件 **基于队列与消息传递技术** 提供同步或异步 可靠的消息传输的支撑性软件系统

应用场景

异步处理'

示例：注册后发送注册邮件和短信

串行方式

![[Pasted image 20250814104727.png]]

并行方式

![[Pasted image 20250814104800.png]]

消息队列

![[Pasted image 20250814104831.png]]

应用解耦

下单时订单系统与库存系统的数据对齐

普通做法

耦合严重 库存系统挂掉 订单系统也挂 ![[Pasted image 20250814105021.png]]

消息队列

![[Pasted image 20250814105054.png]]

流量消峰

流量过大 导致应用挂掉

- 一旦消息队列满了 新来的消息全部丢弃

![[Pasted image 20250814105319.png]]

缺点

系统可用性低

MQ 有挂掉的风险

系统复杂度

如何保证消息丢失 消息重复 消息传递的顺序性

一致性处理

由于消息队列具有延后性 无法第一时间得知写入是否成功 如果失败则导致数据不对齐

常用消息中间件

AMQP (Advanced Message Queuing Protocol)

提供统一消息服务的应用层标准高级消息队列协议 不受制于语言 客户端 中间件

JMS (Java Message Service)

java平台中关于面向消息中间件(MOM)的API,对于在两个应用程序之间 分布式系统中发送消息 进行异步通信 无关具体平台

二者比较

AMQP	JMS
规定协议	规定接口
不限定语言	限定java
模型更加丰富	规定两种消息模型

常见产品

ActiveMQ

- 单机吞吐量达到万级
- 无法处理高并发

RabbitMQ

- 单机吞吐量万级
- 无法处理极高并发
- 使用镜像集群模式 保证高可用
- 消息可靠性上 保证数据不丢失
- 支持消息中间件的高级功能

RocketMQ

- 高吞吐量达到十万级
- 承担互联网高并发
- 分布式框架 可搭建大规模集群 性能高
- 支持高级功能
- 商用收费

Kafka

- 并发强悍
- 支持分布式集群部署
- 保存消息先存到磁盘缓冲区 容易造成缓冲区数据丢失(机器故障)

- 用于制作日志收集(大数据分析) ![[Pasted image 20250814111515.png]]

RabbitMQ

erlang语言开发 基于AMQP协议实现消息队列

集群架构

主备模式

提供主备节点 主节点故障后,备节点提供主节点的读写功能,原来的主节点成为备节点

![[Pasted image 20250815102823.png]]

远程模式

当用户本地的MQ过载时 消息自动转发到其他MQ 从而分摊服务压力 ![[Pasted image 20250815103012.png]]

镜像模式

2-3个节点实现数据同步 消息发送到主节点上 通过mirror队列把数据同步到其他MQ兄弟啊 ![[Pasted image 20250815102259.png]]

多活模式

???? ![[Pasted image 20250815103352.png]]

特点

- 采用经典的**mirror镜像模式**,保证数据100%不丢失
- 主要是数据的同步,一般来讲2-3个节点试下数据同步 一般采用3个

工作原理

基本结构

- 客户端Clients && 生产者Provider
- 服务器
 - 交换机 Exchanges
 - 队列 Queues
- 客户端Clients && 消费者 COnsumer ![[Pasted image 20250815104106.png]]
- **Broker**:消息队列服务进程
 - **Exchange**:消息队列交换机 按一定规则将消息转发到某个队列
 - **Queue**:消息队列 存储消息的队列 消息到达队列并转发给指定消费者
- **Producer**:消息生产者 生产客户端消息

1. 与Broker建立**TCP连接**
2. 与Broker建立通道
3. 通过通道消息发送给Broker
4. Exchange将消息转发给Queue

- **Consumer:**消息消费者 接收MQ转发的消息

1. 与Broker建立**TCP连接**
2. 与Broker建立通道
3. 监听指定的Queue队列
4. 消息到达Queue时 Broker将消息默认推送给消费者
5. 接收消息
6. ack回复

交换机类型

Direct exchange(直连交换机)

1. 队列绑定到某个交换机 同时赋予此次绑定一个路由键
2. 路由值为R的消息,直连交换机将其发送给绑定值为R的队列

```
import org.springframework.amqp.core.Binding;
import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.DirectExchange;
import org.springframework.amqp.core.Queue;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class DirectRabbitConfig {

    //队列 起名: TestDirectQueue
    @Bean
    public Queue TestDirectQueue() {
        // durable:是否持久化,默认是false,持久化队列:会被存储在磁盘上,当消息代理重启时仍然存在,暂存队列:当前连接有效
        // exclusive:默认也是false,只能被当前创建的连接使用,而且当连接关闭后队列即被删除。此参考优先级高于durable
        // autoDelete:是否自动删除,当没有生产者或者消费者使用此队列,该队列会自动删除。
        // return new Queue("TestDirectQueue",true,true,false);

        //一般设置一下队列的持久化就好,其余两个就是默认false
        return new Queue("TestDirectQueue",true);
    }

    //Direct交换机 起名: TestDirectExchange
    @Bean
    DirectExchange TestDirectExchange() {
        // durable / autoDelete
        // return new DirectExchange("TestDirectExchange",true,true);
        return new DirectExchange("TestDirectExchange",true,false);
    }
}
```

```

    }

    //绑定
    //将队列和交换机绑定，并设置用于匹配键：TestDirectRouting
    @Bean
    Binding bindingDirect() {
        //return new Binding("queue", Binding.DestinationType.QUEUE, "DirectExchange",
        "queue", null);
        return
        BindingBuilder.bind(TestDirectQueue()).to(TestDirectExchange()).with("TestDirectRo
        uting");
    }

    @Bean
    DirectExchange lonelyDirectExchange() {
        return new DirectExchange("lonelyDirectExchange");
    }
}

```

Fanout exchange(扇形交换机)

1. 绑定N个队列
2. 接收消息后,消息将发送给N个所有绑定的队列

```

import org.springframework.amqp.core.Binding;
import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.FanoutExchange;
import org.springframework.amqp.core.Queue;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class FanoutRabbitConfig {

    /**
     * 创建三个队列 : fanout.A fanout.B fanout.C
     * 将三个队列都绑定在交换机 fanoutExchange 上
     * 因为是扇型交换机，路由键无需配置,配置也不起作用
     */

    @Bean
    public Queue queueA() {
        return new Queue("fanout.A");
    }

    @Bean
    public Queue queueB() {
        return new Queue("fanout.B");
    }
}

```

```

@Bean
public Queue queueC() {
    return new Queue("fanout.C");
}

@Bean
FanoutExchange fanoutExchange() {
    return new FanoutExchange("fanoutExchange");
}

@Bean
Binding bindingExchangeA() {
    return BindingBuilder.bind(queueA()).to(fanoutExchange());
}

@Bean
Binding bindingExchangeB() {
    return BindingBuilder.bind(queueB()).to(fanoutExchange());
}

@Bean
Binding bindingExchangeC() {
    return BindingBuilder.bind(queueC()).to(fanoutExchange());
}
}

```

Topic exchange(主题交换机)

1. 队列通过路由键绑定交换机
2. 根据消息里的路由值 将消息发给一个或多个绑定队列

```

import org.springframework.amqp.core.Binding;
import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.Queue;
import org.springframework.amqp.core.TopicExchange;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class TopicRabbitConfig {
    //绑定键
    public final static String man = "topic.man";
    public final static String woman = "topic.woman";

    @Bean
    public Queue firstQueue() {
        return new Queue(TopicRabbitConfig.man);
    }

    @Bean

```

```

public Queue secondQueue() {
    return new Queue(TopicRabbitConfig.woman);
}

@Bean
TopicExchange exchange() {
    return new TopicExchange("topicExchange");
}

//将firstQueue和topicExchange绑定,而且绑定的键值为topic.man
//这样只要是消息携带的路由键是topic.man,才会分发到该队列
@Bean
Binding bindingExchangeMessage() {
    return BindingBuilder.bind(firstQueue()).to(exchange()).with(man);
}

//将secondQueue和topicExchange绑定,而且绑定的键值为用上通配路由键规则topic.#
// 这样只要是消息携带的路由键是以topic.开头,都会分发到该队列
@Bean
Binding bindingExchangeMessage2() {
    return BindingBuilder.bind(secondQueue()).to(exchange()).with("topic.#");
}
}

```

路由值

当一个队列的绑定键为# --->接收所有消息 绑定键中未出现#*,则实现直连交换机 借助以上 主题交换机 则可实现直连交换机与扇形交换机 ![[Pasted image 20250815105901.png]]

代码实现

1. 导入依赖

```

<!--rabbitmq-->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>

```

2. yml配置文件

```
server:
  port: 8021
spring:
  #给项目来个名字
  application:
    name: rabbitmq-provider
  #配置rabbitMq 服务器
  rabbitmq:
    host: 127.0.0.1
    port: 5672
    username: root
    password: root
    #虚拟host 可以不设置,使用server默认host
    virtual-host: JCcccHost
```

监听器

单个监听器

```
@Component
public class ListenerA {
    // 正确: 直接在方法上使用 @RabbitListener
    @RabbitListener(queues = "test-queue")
    public void onMessage(Map message) {
        System.out.println("收到消息: " + message);
    }
}
```

多个监听器

```
@Component
@RabbitListener(queues = "test-queue") // 类级别声明监听队列
public class ListenerA {

    // 处理消息的方法 (方法名任意)
    @RabbitHandler
    public void processMessage(Map message) {
        System.out.println("ListenerA 处理消息: " + message);
    }

    // 可添加其他处理不同类型消息的方法
    @RabbitHandler
    public void processTextMessage(String message) {
        System.out.println("处理文本消息: " + message);
    }
}
```


多队列监听

```
@Component
public class ListenerA {

    // 监听多个队列的写法
    @RabbitListeners({
        @RabbitListener(queues = "queue1"),
        @RabbitListener(queues = "queue2")
    })
    public void handleMultiQueue(Map message) {
        System.out.println("从多个队列接收: " + message);
    }
}
```

消息回调(消息确认)

```
server:
  port: 8021
spring:
  #给项目来个名字
  application:
    name: rabbitmq-provider
  #配置rabbitMq 服务器
  rabbitmq:
    host: 127.0.0.1
    port: 5672
    username: root
    password: root
    #虚拟host 可以不设置,使用server默认host
    virtual-host: JCcccHost
    #消息确认配置项

    #确认消息已发送到交换机(Exchange)
    publisher-confirms: true
    #确认消息已发送到队列(Queue)
    publisher-returns: true
```

ConfirmCallback

- 未找到交换机时
- 未找到队列
- 消息推送成功时

ReturnCallback

- 未找到队列

```
import org.springframework.amqp.core.Message;
import org.springframework.amqp.rabbit.connection.ConnectionFactory;
import org.springframework.amqp.rabbit.connection.CorrelationData;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RabbitConfig {

    @Bean
    public RabbitTemplate createRabbitTemplate(ConnectionFactory
connectionFactory){
        RabbitTemplate rabbitTemplate = new RabbitTemplate();
        rabbitTemplate.setConnectionFactory(connectionFactory);
        //设置开启Mandatory,才能触发回调函数,无论消息推送结果怎么样都强制调用回调函数
        rabbitTemplate.setMandatory(true);

        rabbitTemplate.setConfirmCallback(new RabbitTemplate.ConfirmCallback() {
            @Override
            public void confirm(CorrelationData correlationData, boolean ack,
String cause) {
                System.out.println("ConfirmCallback:    "+"相关数
据: "+correlationData);
                System.out.println("ConfirmCallback:    "+"确认情况: "+ack);
                System.out.println("ConfirmCallback:    "+"原因: "+cause);
            }
        });

        rabbitTemplate.setReturnCallback(new RabbitTemplate.ReturnCallback() {
            @Override
            public void returnedMessage(Message message, int replyCode, String
replyText, String exchange, String routingKey) {
                System.out.println("ReturnCallback:    "+"消息: "+message);
                System.out.println("ReturnCallback:    "+"回应码: "+replyCode);
                System.out.println("ReturnCallback:    "+"回应信息: "+replyText);
                System.out.println("ReturnCallback:    "+"交换机: "+exchange);
                System.out.println("ReturnCallback:    "+"路由键: "+routingKey);
            }
        });
        return rabbitTemplate;
    }
}
```

basic.ack

用于肯定确认 deliveryTag 当前消息的唯一id

basic.nack

用于否定确认 deliveryTag 当前消息的唯一id

```
channel.basicNack(deliveryTag, false, true);  
//args-one:是否针对多条消息 true: 当前通道消息id小于当前消息的都拒绝  
//args-two: 是否重新入队
```

basic.reject

用于否定确认 单次仅拒绝单条消息 deliveryTag 当前消息的唯一id

```
`channel.basicReject(deliveryTag, true)`;  
//args-1:是否重新入队
```

```
import com.elegant.rabbitmqconsumer.receiver.MyAckReceiver;  
import org.springframework.amqp.core.AcknowledgeMode;  
import org.springframework.amqp.core.Queue;  
import org.springframework.amqp.rabbit.connection.CachingConnectionFactory;  
import org.springframework.amqp.rabbit.listener.SimpleMessageListenerContainer;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.context.annotation.Bean;  
import org.springframework.context.annotation.Configuration;  
  
@Configuration  
public class MessageListenerConfig {  
  
    @Autowired  
    private CachingConnectionFactory connectionFactory;  
    @Autowired  
    private MyAckReceiver myAckReceiver;//消息接收处理类  
  
    @Bean  
    public SimpleMessageListenerContainer simpleMessageListenerContainer() {  
        SimpleMessageListenerContainer container = new  
SimpleMessageListenerContainer(connectionFactory);  
        container.setConcurrentConsumers(1);  
        container.setMaxConcurrentConsumers(1);  
        // RabbitMQ默认是自动确认, 这里改为手动确认消息  
        container.setAcknowledgeMode(AcknowledgeMode.MANUAL);  
        //设置一个队列  
        container.setQueueNames("TestDirectQueue");  
        //如果同时设置多个如下: 前提是队列都是必须已经创建存在的  
        //  
        container.setQueueNames("TestDirectQueue", "TestDirectQueue2", "TestDirectQueue3");  
  
        //另一种设置队列的方法, 如果使用这种情况, 那么要设置多个, 就使用addQueues  
        //container.setQueues(new Queue("TestDirectQueue", true));  
    }  
}
```

```

        //container.addQueues(new Queue("TestDirectQueue2",true));
        //container.addQueues(new Queue("TestDirectQueue3",true));
        container.setMessageListener(myAckReceiver);

        return container;
    }
}

```

配置消息回调

```

import com.rabbitmq.client.Channel;
import org.springframework.amqp.core.Message;
import org.springframework.amqp.rabbit.listener.api.ChannelAwareMessageListener;
import org.springframework.stereotype.Component;
import java.util.HashMap;
import java.util.Map;

@Component

public class MyAckReceiver implements ChannelAwareMessageListener {

    @Override
    public void onMessage(Message message, Channel channel) throws Exception {
        long deliveryTag = message.getMessageProperties().getDeliveryTag();
        try {
            //因为传递消息的时候用的map传递,所以将Map从Message内取出需要做些处理
            String msg = message.toString();
            String[] msgArray = msg.split("'");//可以点进Message里面看源码,单引号直
接的数据就是我们的map消息数据
            Map<String, String> msgMap = mapStringToMap(msgArray[1].trim(),3);
            String messageId=msgMap.get("messageId");
            String messageData=msgMap.get("messageData");
            String createTime=msgMap.get("createTime");
            System.out.println(" MyAckReceiver messageId:"+messageId+"
messageData:"+messageData+" createTime:"+createTime);
            System.out.println("消费的主题消息来
自: "+message.getMessageProperties().getConsumerQueue());
            channel.basicAck(deliveryTag, true); //第二个参数,手动确认可以被批处理,
当该参数为 true 时,则可以一次性确认 delivery_tag 小于等于传入值的所有消息
// channel.basicReject(deliveryTag, true);//第二个参数, true会重新放回队
列,所以需要自己根据业务逻辑判断什么时候使用拒绝
        } catch (Exception e) {
            channel.basicReject(deliveryTag, false);
            e.printStackTrace();
        }
    }

    // {key=value,key=value,key=value} 格式转换成map
    private Map<String, String> mapStringToMap(String str,int entryNum ) {
        str = str.substring(1, str.length() - 1);
    }
}

```

```
String[] strs = str.split(",",entryNum);
Map<String, String> map = new HashMap<String, String>();
for (String string : strs) {
    String key = string.split("=")[0].trim();
    String value = string.split("=")[1];
    map.put(key, value);
}
return map;
}
```